UNIVERSIDAD NACIONAL DE SAN AGUSTIN

ESCUELA PROFESIONAL DE INGENIERIA DE SISTEMAS

# Laboratorio:

Templates en C++

## I. OBJETIVOS

- Conocer el uso de las plantillas.

## II. TEMAS A TRATAR

- Templates (plantillas) en C++.

## III. MARCO TEORICO

Las plantillas o templates es una característica especial del lenguaje de programación C++, hacen posible usar una función o clase para manejar muchos tipos de datos diferentes, sin tener que especificar el tipo de todos o alguno de sus miembros.
Los templates no son obligatorios en la programación orientada a objetos, pero brindan la característica de evitar reescribir código/estructuras para adaptar a otro escenario de problema similar.

Notación para declarar:

- Plantilla de funciones

Opción 1
*template <typename identifier> function_definition*

Opción 2
*template <class identifier>*

- Plantilla de clases

*template <class identifier> class_definition;*

```
template <class identifier>
    return_type name_class<identifier>::name_function(argument){
   // Implemantation
}
```

## IV. ACTIVIDADES

a) Crear una función plantilla que permita realizar la aritmética de 2 números sin importar el tipo input (entero, flotante, doble, o similar)

```
template<typename mitipo>
    mitipo aritmetica(mitipo x, mitipo y) {
    return x + y;
}


 //invocar al template
 aritmetica<int>(10, 45);

 aritmetica<double>(3.45, 8.10);

 aritmetica<float>(0.14f, 0.147f);
```

b) Crear una función plantilla que devuelva el caracter 'mayor' (según orden alfabetico), o el numero mayor sin importar el tipo input.

```
char mayorLetras(char a, char b) {
    return (a > b ? a : b);
}

int mayorNumeros(int a, int b) {
    return (a > b ? a : b);
}

template <typename Tipo> Tipo mayorDe(Tipo a, Tipo b) {
    return (a > b ? a : b);
}


 //invocar a funcion template de mayor de
 char x = mayorDe<char>('e', 'z');
 int y = mayorDe<int>(9, 4);

 cout << "caracter mayor: " << x << endl;
 cout << "numero mayor: " << y << endl;
```

c) Clase plantilla para insertar cualquier tipo de dato numérico en una posición específica de un array.

```
template<typename T>
class Contenedor {
public:
    T metodo1(int xx) {
        return array[xx];
    }
    void metodo2(T val, int i) {
        array[i] = val;
    }

    void mostrar() const;
private:
    T array[4];
};


template<typename T>
void Contenedor<T>::mostrar() const {
    for (int i = 0; i <= 4; i++) {
        cout << array[i] << " ";

    }
};


 //invocar a clase template
 Contenedor<int> z;
 z.metodo2(10, 0);
 z.metodo2(50, 1);
 z.metodo2(60, 2);
 z.metodo2(70, 3);

 z.mostrar();
```

d) Clase plantilla para estructuras pila, permitir manipular inserciones de tipos numéricos o caracteres.

```
template<class T>
class Pila {
public:
    Pila(int nelem = 10);
    void insertar(T);
    void mostrarPila();
private:
    int nelementos;
    T* cadena;
    int limite;
};
```

```
template<class T> Pila<T>::Pila(int nelem) {
    nelementos = nelem;
    cadena = new T(nelementos);
    limite = 0;
}

//funciones miembro
template<class T> void Pila<T>::insertar(T elem) {
    if (limite < nelementos)
        cadena[limite++] = elem;
};

template<class T> void Pila<T>::mostrarPila() {

    for (int i = 0; i < limite; i++) {
        std::cout << cadena[i] << ", ";
    }
}
```

//Llamada desde algún método, o desde un main. Todo el código anterior puede estar en un header.

```
Pila<int> pila1(4);
pila1.insertar(1);
pila1.insertar(3);
pila1.mostrarPila();
```

```
Pila<char> pila2(4);
pila2.insertar('a');
pila2.insertar('b');
pila2.insertar('c');
pila2.mostrarPila();
```

## VII. Ejercicios

1. Usar clase template para implementar estructuras de lista enlazada simple, que permita armar secuencia de edades, secuencia de palabras.
2. Trascribir y analizar el siguiente código, describir el comportamiento.

```
#include <iostream>
using namespace std;

template <class T>
class Contendor {
    T elemento;
public:
    Contendor(T arg) { elemento = arg; }
    T add() { return ++elemento; }
};

template <>
class Contendor <char> {
    char elemento;
public:
    Contendor(char arg) { elemento = arg; }
    char uppercase()
    {
        if ((elemento >= 'a') && (elemento <= 'z')) {
            elemento += 'A' - 'a';
        }
        return elemento;
    }
};
```

```
int main() {
    Contendor<int> cint(5);
    Contendor<char> cchar('t');
    std::cout << cint.add() << std::endl;
    std::cout << cchar.uppercase() << std::endl;
    return 0;
}
```

3. Trascribir y analizar el siguiente código, describir el comportamiento.

```
#include <iostream>
using namespace std;

template <class T = char, int N = 8>
class Comun1 {
    T bloque[N];
public:
    void set(int num, T tval);
    T get(int num);
};

template <class T, int N>
void Comun1<T, N>::set(int num, T tval) {
    bloque[num] = tval;
}

template <class T, int N>
T Comun1<T, N>::get(int num) {
    return bloque[num];
}

int main() {
    Comun1 <int, 5> objInt;
    Comun1 <double, 5> objFloat;
    Comun1 < > obj;
    objInt.set(0, 10);
    objFloat.set(2, 3.1);

    std::cout << objInt.get(0) << std::endl;
    std::cout << objFloat.get(2) << std::endl;
    std::cout << obj.get(4) << std::endl;

    return 0;
}
```

## VII. Cuestionario

1. Revisar y encontrar diferencias, ventajas y desventajas entre plantillas y polimorfismo.
2. Revisar sobre funciones amigas, cómo se integra a las plantillas.

**Referencias:**


- Ceballos, F.J., programación orientada a Objetos con C++, 2da ed.,1997.
- Robert Lafore, Object-Oriented programamming in C++, fourth edition.